

香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

Operating Systems
Lecture 9

Synchronization: semaphores and monitors

Prof. Yunming XIAO

School of Data Science

Acknowledgments: Ion Stoica and Matei Zaharia, Berkeley CS 162

Higher-level primitives than locks

- Goal of last couple of lectures:
  - What is right abstraction for synchronizing threads that share memory?
  - Want as high-level a primitive as possible

- Synchronization is a way of coordinating multiple concurrent activities that are using shared state
  - This lecture and the next present some ways of structuring sharing

Recall: simple locks with test&set atomic instruction

- Simple lock that doesn't require entry into the kernel

```
// Storage: just a memory location in userspace; 0 means "unlocked"
int mylock = 0;  // Interface: acquire(&mylock);
                 //            release(&mylock);

acquire(int *thelock) {
    while (test&set(thelock));    // Atomic operation!
}

release(int *thelock) {
    *thelock = 0;                 // Atomic operation!
}
```

Better locks using test&set

- Idea: only busy-wait to atomically check lock value



```
int guard = 0; // Global Variable!
int mylock = FREE; // Interface: acquire(&mylock);
                   //            release(&mylock);

acquire(int *thelock) {
  // Short busy-wait time
  while (test&set(guard));
  if (*thelock == BUSY) {
    put thread on wait queue;
    go to sleep() & guard = 0;
    // guard == 0 on wakeup!
  } else {
    *thelock = BUSY;
    guard = 0;
  }
}
```

```
release(int *thelock) {
  // Short busy-wait time
  while (test&set(guard));
  if anyone on wait queue {
    take thread off wait queue
    Place on ready queue;
  } else {
    *thelock = FREE;
  }
  guard = 0;
}
```

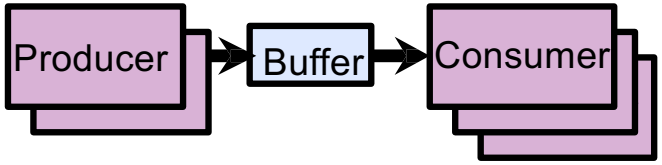
Linux futex: interface to kernel sleep functionality

```
#include <linux/futex.h>
#include <sys/time.h>

int futex(int *uaddr, int futex_op, int val,
          const struct timespec *timeout);
```

- uaddr points to a 32-bit value in user space
- futex_op
  - FUTEX_WAIT – if *uaddr == val, sleep till FUTEX_WAKE
    - ***Atomic*** check that condition still holds after we disable interrupts (in kernel!)
  - FUTEX_WAKE – wake up at most val waiting threads
  - FUTEX_WAKE_OP, FUTEX_LOCK_PI, FUTEX_CMP_REQUEUE: More interesting operations!
- timeout
  - ptr to a timespec structure that specifies a timeout for the op

Producer-consumer with a bounded buffer

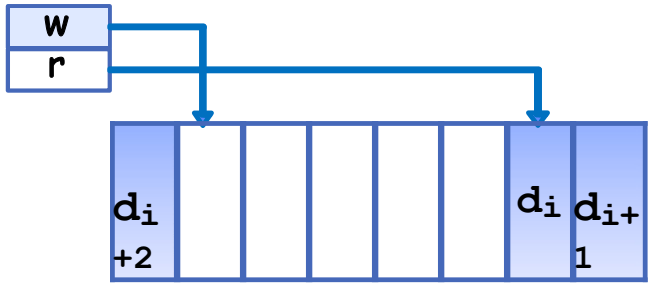


- Problem Definition
  - Producer(s) put things into a shared buffer
  - Consumer(s) take them out
  - Need synchronization to coordinate producer/consumer

- Don't want producer and consumer to have to work in lockstep, so put a fixed-size buffer between them
  - Need to synchronize access to this buffer
  - Producer needs to wait if buffer is full
  - Consumer needs to wait if buffer is empty

Circular buffer data structure (sequential case)

```
typedef struct buf {
  int write_index;
  int read_index;
  <type> *entries[BUFSIZE];
} buf_t;
```



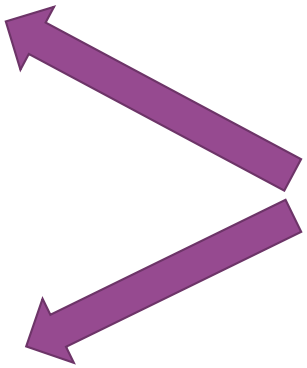
- Insert: write & bump write ptr (enqueue)
- Remove: read & bump read ptr (dequeue)
- How to tell if full (on insert) or empty (on remove)?
- And what do you do if it is?
- What needs to be atomic?

Circular buffer – first cut

```
Mutex buf_lock = <initially unlocked>

Producer(item) {
  acquire(&buf_lock);
  while (buffer full) {};  // Wait for a free slot
  enqueue(item);
  release(&buf_lock);
}

Consumer() {
  acquire(&buf_lock);
  while (buffer empty) {};
  item = dequeue();
  release(&buf_lock);
  return item;
}
```



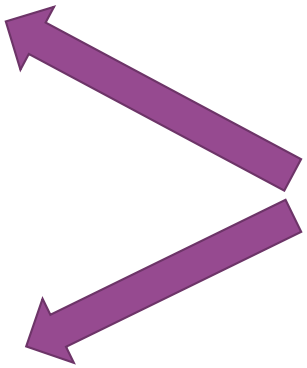
- Will we ever come out of the wait loop?

Circular buffer – second cut

```
Mutex buf_lock = <initially unlocked>

Producer(item) {
  acquire(&buf_lock);
  while (buffer full) { release(&buf_lock); acquire(&buf_lock); }
  enqueue(item);
  release(&buf_lock);
}

Consumer() {
  acquire(&buf_lock);
  while (buffer empty) { release(&buf_lock); acquire(&buf_lock); };
  item = dequeue();
  release(&buf_lock);
  return item;
}
```



- What happens when one is waiting for the other?
  - Multiple cores?
  - Single core?

Semaphores

- Semaphores are a type of generalized lock
- First defined by Dijkstra in late 60s
- Main synchronization primitive in original UNIX



Semaphores

- A Semaphore has a non-negative integer value and supports the following operations:
  - Set value when you initialize
  - Down() or P(): an atomic operation that waits for semaphore to become positive, then decrements it by 1
  - Up() or V(): an atomic operation that increments the semaphore by 1, waking up a waiting P, if any

Semaphores like integers except…

- Semaphores are like integers, except:
  - No negative values
  - Only operations allowed are P and V – can't read or write value, except initially
  - Operations must be atomic
    - Two P's together can't decrement value below zero
    - Thread going to sleep in P won't miss wakeup from V – even if both happen at the same time

Two uses of semaphores

- Mutual Exclusion (initial value = 1)
- Also called “Binary Semaphore” or “mutex”.
- Can be used for mutual exclusion, just like a lock:

```
semaP(&mysem);
// Critical section goes here
semaV(&mysem);
```

Two uses of semaphores

- Scheduling constraints (initial value = 0)
- Allow thread 1 to wait for a signal from thread 2
  - thread 2 schedules thread 1 when a given event occurs
- Suppose you had to implement ThreadJoin which must wait for thread to terminate

```
// Initial value of semaphore = 0
ThreadJoin {
  semaP(&mysem);
}
ThreadFinish {
  semaV(&mysem);
}
```

Bounded buffer: correctness constraints for solution

- Correctness constraints:
  - Consumer must wait for producer to fill buffers, if none full (scheduling constraint)
  - Producer must wait for consumer to empty buffers, if all full (scheduling constraint)
  - Only one thread can manipulate buffer queue at a time (mutual exclusion)

Bounded buffer: correctness constraints for solution

- General rule of thumb: use a separate semaphore for each constraint

  - Semaphore `full_buffers`; // consumer's constraint
  - Semaphore `empty_buffers`; // producer's constraint
  - Semaphore `mutex`; // mutual exclusion

Let's drink coke!




```
Semaphore full_slots = 0;          // Initially, no coke
Semaphore empty_slots = buf_size; // Initially, num empty slots
Semaphore mutex = 1;               // No one using machine

Producer(item) {
  semaP(&empty_slots);             // Wait until space




}
Consumer() {
  semaP(&full_slots);              // Check if there's a coke




}
```

Let's drink coke!




```
Semaphore full_slots = 0;          // Initially, no coke
Semaphore empty_slots = buf_size; // Initially, num empty slots
Semaphore mutex = 1;               // No one using machine

Producer(item) {
  semaP(&empty_slots);             // Wait until space

  enqueue(item);


}
Consumer() {
  semaP(&full_slots);              // Check if there's a coke

  item = dequeue();



}
```

Let's drink coke!




```
Semaphore full_slots = 0;          // Initially, no coke
Semaphore empty_slots = buf_size; // Initially, num empty slots
Semaphore mutex = 1;               // No one using machine

Producer(item) {
  semaP(&empty_slots);             // Wait until space

  enqueue(item);

  semaV(&full_slots);              // Tell consumers there is more coke
}
Consumer() {
  semaP(&full_slots);              // Check if there's a coke

  item = dequeue();

  semaV(&empty_slots);             // Tell producer need more
  return item
}
```

Let's drink coke!




```
Semaphore full_slots = 0;          // Initially, no coke
Semaphore empty_slots = buf_size; // Initially, num empty slots
Semaphore mutex = 1;               // No one using machine

Producer(item) {
  semaP(&empty_slots);             // Wait until space
  semaP(&mutex);                   // Wait until machine free
  enqueue(item);
  semaV(&mutex);
  semaV(&full_slots);              // Tell consumers there is more coke
}
Consumer() {
  semaP(&full_slots);              // Check if there's a coke
  semaP(&mutex);                   // Wait until machine free
  item = dequeue();
  semaV(&mutex);
  semaV(&empty_slots);             // Tell producer need more
  return item
}
```

Let's drink coke!



```
Semaphore full_slots = 0;          // Initially, no coke
Semaphore empty_slots = buf_size; // Initially, num empty slots
Semaphore mutex = 1;               // No one using machine

Producer(item) {
  semaP(&empty_slots);             // Wait until space
  semaP(&mutex);                   // Wait until machine free
  enqueue(item);
  semaV(&mutex);
  semaV(&full_slots);              // Tell consumers there is
}                                  // more coke
Consumer() {
  semaP(&full_slots);              // Check if there's a coke
  semaP(&mutex);                   // Wait until machine free
  item = dequeue();
  semaV(&mutex);
  semaV(&empty_slots);             // Tell producer need more
  return item
}
```

full_slots signal space

empty_slots signal space

Critical sections using mutex protect integrity of the queue

Discussion about solution

- Why asymmetry?
  - Producer does: **semaP(&empty_buffer), semaV(&full_buffer)**
  - Consumer does: **semaP(&full_buffer), semaV(&empty_buffer)**
  - Does order matter? What if we decrement mutex before full/empty_buffer?

Let's drink coke!



```
Semaphore full_slots = 0;          // Initially, no coke
Semaphore empty_slots = buf_size; // Initially, num empty slots
Semaphore mutex = 1;               // No one using machine

Producer(item) {
  semaP(&mutex);                   // Wait until machine free
  semaP(&empty_slots);             // Wait until space
  enqueue(item);
  semaV(&full_slots);              // Tell consumers there is more coke
  semaV(&mutex);
}
Consumer() {
  semaP(&mutex);                   // Wait until machine free
  semaP(&full_slots);              // Check if there's a coke
  item = dequeue();
  semaV(&empty_slots);             // Tell producer need more
  semaV(&mutex);
  return item
}
```

Let's drink coke!




```
Semaphore full_slots = 0;          // Initially, no coke
Semaphore empty_slots = buf_size; // Initially, num empty slots
Semaphore mutex = 1;               // No one using machine

Producer(item) {
  semaP(&empty_slots);             // Wait until space
  semaP(&mutex);                   // Wait until machine free
  enqueue(item);
  semaV(&full_slots);              // Tell consumers there is more coke
  semaV(&mutex);
}
Consumer() {
  semaP(&full_slots);              // Check if there's a coke
  semaP(&mutex);                   // Wait until machine free
  item = dequeue();
  semaV(&empty_slots);             // Tell producer need more
  semaV(&mutex);
  return item
}
```

Semaphores are good but…

- Semaphores are a huge step up; just think of trying to do the bounded buffer with only loads and stores or even with locks!

- Problem is that semaphores are dual purpose:
  - They are used for both mutex and scheduling constraints
  - Example: the fact that flipping of P's in bounded buffer gives deadlock is not immediately obvious. How do you prove correctness to someone?

End of recall

Where are we going with synchronization?

Programs	Shared Programs	
Higher-level API	Locks Monitors	Semaphores Send/Receive
Hardware	Load/Store Test&Set	Disable Ints Compare&Swap

- We are going to implement various higher-level synchronization primitives using atomic operations

Monitors

Monitors: an even nicer abstraction!

- Use *locks* for mutual exclusion and *condition variables* for scheduling constraints

- **Monitor**: a lock and zero or more condition variables for managing concurrent access to shared data

- A monitor is a paradigm for concurrent programming
  - Some languages, like Java, provide them natively
  - Others use actual locks and condition var objects

Condition variables

- A queue of threads waiting for something (a condition) *inside* a critical section

- Key idea: allow going to sleep inside the critical section by atomically releasing lock at time we go to sleep

- Contrast to semaphores: can't wait inside critical section

Condition variables

- Operations on condition variables:
  - `Wait(&lock)`: Atomically release lock and go to sleep. Re-acquire lock later, when returning from sleep.
  - `Signal()`: Wake up one waiter, if any exists.
  - `Broadcast()`: Wake up all waiters.

- Rule: must hold lock when doing condition variable ops!

Monitor with condition variables

- Lock: the lock provides mutual exclusion to shared data
  - Always acquire before accessing shared data structures
  - Always release after finishing with shared data
  - Lock is initially free

- Condition variable: a queue of threads waiting for something inside a critical section
  - Key idea: make it possible to go to sleep inside critical section by atomically releasing lock as we go to sleep

Infinite synchronized buffer (with condition variable)

```
lock buf_lock;                  // Initially unlocked
condition buf_CV;               // Initially empty
queue queue;                    // Actual queue!
```

Infinite synchronized buffer (with condition variable)

```
lock buf_lock;                      // Initially unlocked
condition buf_CV;                   // Initially empty
queue queue;                        // Actual queue!

Producer(item) {
  acquire(&buf_lock);               // Get Lock
  enqueue(&queue, item);            // Add item
  cond_signal(&buf_CV);             // Signal any waiters
  release(&buf_lock);               // Release Lock
}
```

Infinite synchronized buffer (with condition variable)

```
lock buf_lock;                          // Initially unlocked
condition buf_CV;                       // Initially empty
queue queue;                            // Actual queue!

Producer(item) {
  acquire(&buf_lock);                   // Get Lock
  enqueue(&queue, item);                // Add item
  cond_signal(&buf_CV);                 // Signal any waiters
  release(&buf_lock);                   // Release Lock
}

Consumer() {
  acquire(&buf_lock);                   // Get Lock
  if (is_empty(&queue)) {
    cond_wait(&buf_CV, &buf_lock);  // If empty, sleep
  }
  item = dequeue(&queue);               // Get next item
  release(&buf_lock);                   // Release lock
  return(item);
}
```

Infinite synchronized buffer (with condition variable)

```
lock buf_lock;                          // Initially unlocked
condition buf_CV;                       // Initially empty
queue queue;                            // Actual queue!

Producer(item) {
  acquire(&buf_lock);                   // Get Lock
  enqueue(&queue, item);                // Add item
  cond_signal(&buf_CV);                 // Signal any waiters
  release(&buf_lock);                   // Release Lock
}

Consumer() {
  acquire(&buf_lock);                   // Get Lock
  while (is_empty(&queue)) {
    cond_wait(&buf_CV, &buf_lock);  // If empty, sleep
  }
  item = dequeue(&queue);               // Get next item
  release(&buf_lock);                   // Release lock
  return(item);
}
```

Mesa vs. Hoare monitors

- Need to be careful about precise definition of signal and wait

```
 while (is_empty(&queue)) {
    cond_wait(&buf_CV, &buf_lock);  // If empty, sleep
  }
item = dequeue(&queue);             // Get next item
```

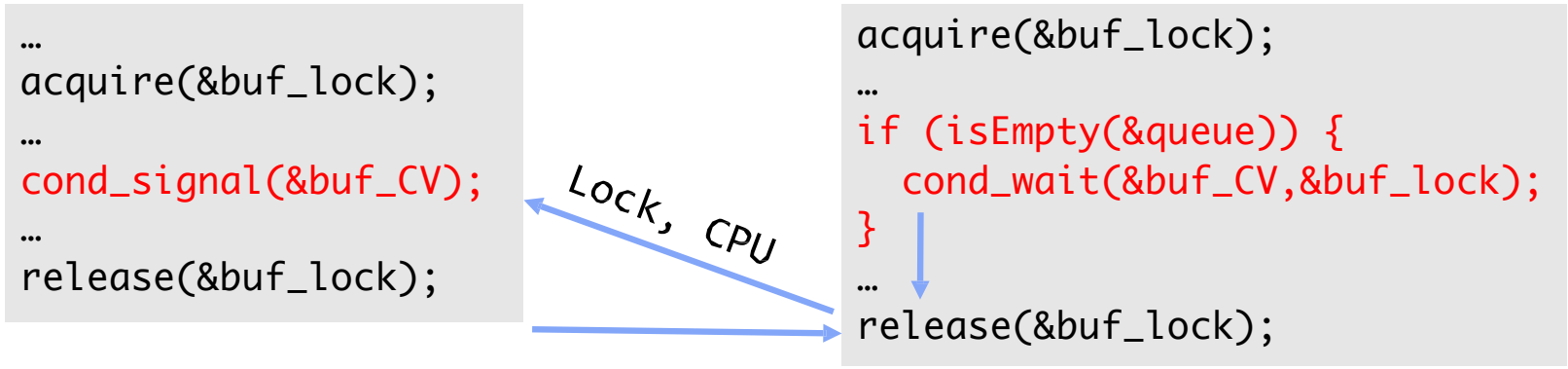
- Why didn't we do this?

```
 if (is_empty(&queue)) {
    cond_wait(&buf_CV, &buf_lock);  // If empty, sleep
  }
item = dequeue(&queue);             // Get next item
```

- Answer: depends on the type of scheduling
  - Mesa-style: named after Xerox-Parc Mesa Operating System
    - Most OSes use Mesa scheduling
  - Hoare-style: named after British logician Tony Hoare

Hoare monitors

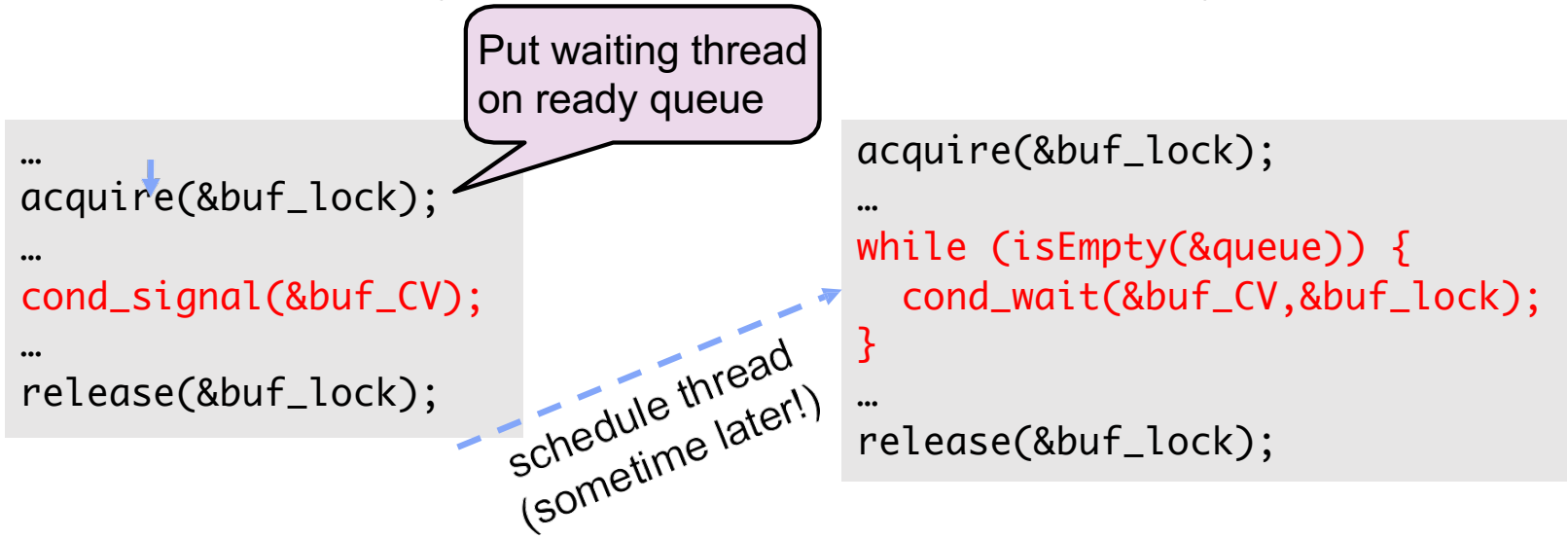
- Signaler gives up lock, CPU to waiter; waiter runs immediately
- Then, waiter gives up lock, processor back to signaler when it exits critical section or if it waits again



- At first glance, this seems like good semantics
- Waiter gets to run immediately, condition is still correct!

Mesa monitors

- Signaler keeps lock and processor
- Waiter placed on ready queue with no special priority



- Practically, need to check condition again after wait
  - By the time the waiter gets scheduled, condition may be false again
  - so, just check again with the “while” loop

Bounded buffer with monitor

```
lock buf_lock = <initially unlocked>
condition is_not_empty = <initially empty>
condition is_not_full = <initially empty>
```

Bounded buffer with monitor

```
lock buf_lock = <initially unlocked>
condition is_not_empty = <initially empty>
condition is_not_full = <initially empty>

Producer(item) {
  acquire(&buf_lock);
  while (buffer full) { cond_wait(&is_not_full, &buf_lock); }
  enqueue(item);
  cond_signal(&is_not_empty);
  release(&buf_lock);
}
```

Bounded buffer with monitor

```
lock buf_lock = <initially unlocked>
condition is_not_empty = <initially empty>
condition is_not_full = <initially empty>

Producer(item) {
  acquire(&buf_lock);
  while (buffer full) { cond_wait(&is_not_full, &buf_lock); }
  enqueue(item);
  cond_signal(&is_not_empty);
  release(&buf_lock);
}

Consumer() {
  acquire(buf_lock);
  while (buffer empty) { cond_wait(&is_not_empty, &buf_lock); }
  item = dequeue();
  cond_signal(&is_not_full);
  release(buf_lock);
  return item;
}
```

Again, why the while loop?

- MESA semantics
- For most operating systems, when a thread is woken up by signal(), it is simply put on the ready queue
- It may or may not reacquire the lock immediately!
  - Another thread could be scheduled first and "sneak in" to empty the queue
  - Need a loop to re-check condition on wakeup
- Is this busy waiting?

Basic structure of *Mesa* monitor program

- Monitors represent the synchronization logic of the program
  - Wait if necessary
  - Signal when change something so any waiting threads can proceed

```
lock
while (need to wait) {
  condvar.wait();
}
unlock
```

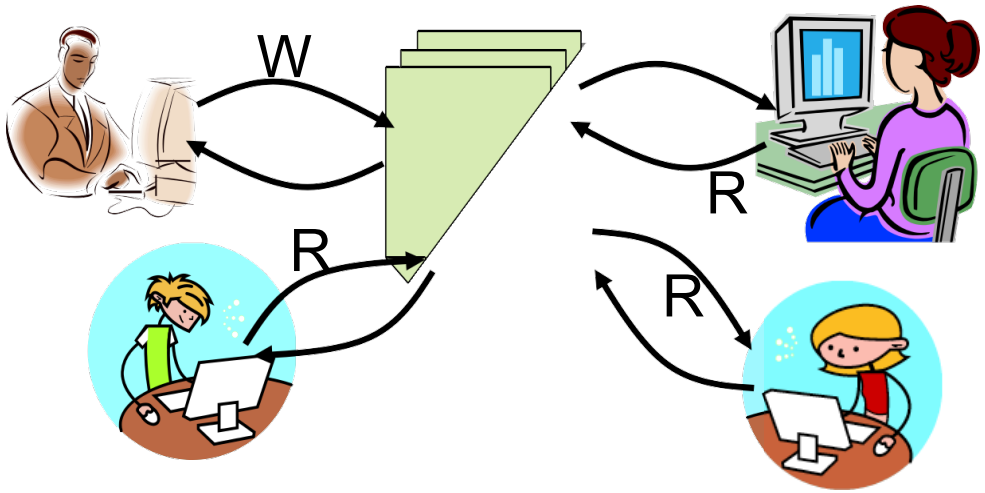
Check and/or update state variables
Wait if neccessary

```
do something so need to wait
```

```
lock
condvar.signal();
unlock
```

Check and/or update state variables

Readers/writers problem



- Motivation: consider a shared database
  - Two classes of users:
    - Readers – never modify database
    - Writers – read and modify database
  - Is using a single lock on the whole database sufficient?
    - Like to have many readers at the same time
    - Only one writer at a time

Basic readers/writers solution

- Correctness Constraints:
  - Readers can access database when no writers
  - Writers can access database when no readers or writers
  - Only one thread manipulates state variables at a time

- Basic structure of a solution:
  - `Reader()`

    Wait until no writers

    Access database

    Check out – wake up a waiting writer
  - `Writer()`

    Wati until no active readers or writers

    Access databases

    Check out – wake up waiting readers or writer

Basic readers/writers solution

- State variables (Protected by a lock called “lock”):
  - int AR: Number of active readers; initially = 0
  - int WR: Number of waiting readers; initially = 0
  - int AW: Number of active writers; initially = 0
  - int WW: Number of waiting writers; initially = 0
  - Condition ok_to_read = NIL
  - Condition ok_to_write = NIL

Code for a reader

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

Code for a writer

```
Writer() {
  // First check self into system
  acquire(&lock);
  while ((AW + AR) > 0) {              // Is it safe to read?
    WW++;                              // No. Writers exit
    cond_wait(&ok_to_write, &lock);    // Sleep on cond var
    WW--;                              // No longer waiting
  }
  AW++;                                // Now we are active!
  release(&lock);
  // Perform actual read/write access
  access_database(read_write);
  // Now, check out of system
  acquire(&lock);
  AW--;                                // No longer active
  if (WW > 0)                          // Give priority to writers
    cond_signal(&ok_to_write);         // Wake up one writer
  else if (WR > 0)                     // Otherwise, wake up readers
    cond_broadcast(&ok_to_read);       // Wake up all readers
  release(&lock);
}
```

Simulation of readers/writers solution

- Use an example to simulate the solution

- Consider the following sequence of operators:
  - R1, R2, W1, R3

- Initially: AR = 0, WR = 0, AW = 0, WW = 0

Simulation of readers/writers solution

R1 comes alone (no waiting threads)
AR = 0, WR = 0, AW = 0, WW = 0

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

Simulation of readers/writers solution

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {                // Is it safe to read?
    WR++;                                // No. Writers exit
    cond_wait(&ok_to_read, &lock);       // Sleep on cond var
    WR--;                                // No longer waiting
  }
  AR++;                                  // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                                  // No longer active
  if (AR == 0 && WW > 0)                 // No other active readers
    cond_signal(&ok_to_write);           // Wake up one writer
  release(&lock);
}
```

R1 comes alone (no waiting threads)
AR = 0, WR = 0, AW = 0, WW = 0

Simulation of readers/writers solution

R1 comes alone (no waiting threads)
AR = 1, WR = 0, AW = 0, WW = 0

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

Simulation of readers/writers solution

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

R1 comes alone (no waiting threads)
AR = 1, WR = 0, AW = 0, WW = 0

Simulation of readers/writers solution

R1 comes alone (no waiting threads)
AR = 1, WR = 0, AW = 0, WW = 0

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

Simulation of readers/writers solution

R2 comes alone (R1 accessing database)
AR = 1, WR = 0, AW = 0, WW = 0

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

Simulation of readers/writers solution

R2 comes alone (R1 accessing database)
AR = 1, WR = 0, AW = 0, WW = 0

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {                // Is it safe to read?
    WR++;                                // No. Writers exit
    cond_wait(&ok_to_read, &lock);       // Sleep on cond var
    WR--;                                // No longer waiting
  }
  AR++;                                  // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                                  // No longer active
  if (AR == 0 && WW > 0)                 // No other active readers
    cond_signal(&ok_to_write);           // Wake up one writer
  release(&lock);
}
```

Simulation of readers/writers solution

R2 comes alone (R1 accessing database)
AR = 2, WR = 0, AW = 0, WW = 0

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

Simulation of readers/writers solution

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

R2 comes alone (R1 accessing database)

AR = 2, WR = 0, AW = 0, WW = 0

Simulation of readers/writers solution

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR =                                              s
    cond_s
  release(
}
```

R2 comes alone (R1 accessing database)
AR = 2, WR = 0, AW = 0, WW = 0

Assume readers take a while to access database
Situation: locks released, only AR is non-zero

Simulation of readers/writers solution

W1 comes alone (R1&R2 accessing database)
AR = 2, WR = 0, AW = 0, WW = 0

```
Writer() {
  // First check self into system
  acquire(&lock);
  while ((AW + AR) > 0) {              // Is it safe to read?
    WW++;                              // No. Writers exit
    cond_wait(&ok_to_write, &lock);    // Sleep on cond var
    WW--;                              // No longer waiting
  }
  AW++;                                // Now we are active!
  release(&lock);
  // Perform actual read/write access
  access_database(read_write);
  // Now, check out of system
  acquire(&lock);
  AW--;                                // No longer active
  if (WW > 0)                          // Give priority to writers
    cond_signal(&ok_to_write);         // Wake up one writer
  else if (WR > 0)                     // Otherwise, wake up readers
    cond_broadcast(&ok_to_read);       // Wake up all readers
  release(&lock);
}
```

Simulation of readers/writers solution

W1 comes alone (R1&R2 accessing database)
AR = 2, WR = 0, AW = 0, WW = 0

```
Writer() {
  // First check self into system
  acquire(&lock);
  while ((AW + AR) > 0) {                  // Is it safe to read?
    WW++;                                  // No. Writers exit
    cond_wait(&ok_to_write, &lock);        // Sleep on cond var
    WW--;                                  // No longer waiting
  }
  AW++;                                    // Now we are active!
  release(&lock);
  // Perform actual read/write access
  access_database(read_write);
  // Now, check out of system
  acquire(&lock);
  AW--;                                    // No longer active
  if (WW > 0)                              // Give priority to writers
    cond_signal(&ok_to_write);             // Wake up one writer
  else if (WR > 0)                         // Otherwise, wake up readers
    cond_broadcast(&ok_to_read);           // Wake up all readers
  release(&lock);
}
```

Simulation of readers/writers solution

W1 comes alone (R1&R2 accessing database)
AR = 2, WR = 0, AW = 0, WW = 1

```
Writer() {
  // First check self into system
  acquire(&lock);
  while ((AW + AR) > 0) {              // Is it safe to read?
    WW++;                              // No. Writers exit
    cond_wait(&ok_to_write, &lock);    // Sleep on cond var
    WW--;                              // No longer waiting
  }
  AW++;                                // Now we are active!
  release(&lock);
  // Perform actual read/write access
  access_database(read_write);
  // Now, check out of system
  acquire(&lock);
  AW--;                                // No longer active
  if (WW > 0)                          // Give priority to writers
    cond_signal(&ok_to_write);         // Wake up one writer
  else if (WR > 0)                     // Otherwise, wake up readers
    cond_broadcast(&ok_to_read);       // Wake up all readers
  release(&lock);
}
```

Simulation of readers/writers solution

```
Writer() {
  // First check self into system
  acquire(&lock);
  while ((AW + AR) > 0) {           // Is it safe to read?
    WW++;                           // No. Writers exit
    cond_wait(&ok_to_write, &lock); // Sleep on cond var
    WW--;                           // No longer waiting
  }
  AW++;                             // Now we are active!
  release(&lock);
  // Perform actual read/write access
  access_database(read_write);
  // Now, check out of system
  acquire(&lock);
  AW--;                             // No longer active
  if (WW > 0)                       // Give priority to writers
    cond_signal(&ok_to_write);      // Wake up one writer
  else if (WR > 0)                  // Otherwise, wake up readers
    cond_broadcast(&ok_to_read);    // Wake up all readers
  release(&lock);
}
```

W1 comes alone (R1&R2 accessing database)

AR = 2, WR = 0, AW = 0, WW = 1

Simulation of readers/writers solution

R3 comes alone (R1 & R2 accessing database, W1 waiting)
AR = 2, WR = 0, AW = 0, WW = 1

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

Simulation of readers/writers solution

R3 comes alone (R1 & R2 accessing database, W1 waiting)
AR = 2, WR = 0, AW = 0, WW = 1

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {                // Is it safe to read?
    WR++;                                // No. Writers exit
    cond_wait(&ok_to_read, &lock);       // Sleep on cond var
    WR--;                                // No longer waiting
  }
  AR++;                                  // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                                  // No longer active
  if (AR == 0 && WW > 0)                 // No other active readers
    cond_signal(&ok_to_write);           // Wake up one writer
  release(&lock);
}
```

Simulation of readers/writers solution

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

R3 comes alone (R1 & R2 accessing database, W1 waiting)

AR = 2, WR = 1, AW = 0, WW = 1

Simulation of readers/writers solution

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

R3 comes alone (R1 & R2 accessing database, W1 waiting)

AR = 2, WR = 1, AW = 0, WW = 1

Simulation of readers/writers solution

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
}
```

R1 & R2 accessing database, W1 and R3 waiting

AR = 2, WR = 1, AW = 0, WW = 1

Status:

- R1 and R2 still reading
- W1 and R3 waiting on ok_to_write and ok_to_read, respectively

Simulation of readers/writers solution

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

R2 finishes (R1 accessing database, W1 and R3 waiting)

AR = 2, WR = 1, AW = 0, WW = 1

Simulation of readers/writers solution

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

R2 finishes (R1 accessing database, W1 and R3 waiting)

AR = 1, WR = 1, AW = 0, WW = 1

Simulation of readers/writers solution

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

R2 finishes (R1 accessing database, W1 and R3 waiting)

AR = 1, WR = 1, AW = 0, WW = 1

Simulation of readers/writers solution

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

R2 finishes (R1 accessing database, W1 and R3 waiting)

AR = 1, WR = 1, AW = 0, WW = 1

Simulation of readers/writers solution

R1 finishes (W1 and R3 waiting)
AR = 1, WR = 1, AW = 0, WW = 1

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {           // Is it safe to read?
    WR++;                           // No. Writers exit
    cond_wait(&ok_to_read, &lock);  // Sleep on cond var
    WR--;                           // No longer waiting
  }
  AR++;                             // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                             // No longer active
  if (AR == 0 && WW > 0)            // No other active readers
    cond_signal(&ok_to_write);      // Wake up one writer
  release(&lock);
}
```

Simulation of readers/writers solution

R1 finishes (W1 and R3 waiting)
AR = 0, WR = 1, AW = 0, WW = 1

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

Simulation of readers/writers solution

R1 finishes (W1 and R3 waiting)
AR = 0, WR = 1, AW = 0, WW = 1

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

Simulation of readers/writers solution

R1 signals a writer (W1 and R3 waiting)
AR = 0, WR = 1, AW = 0, WW = 1

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

Simulation of readers/writers solution

```
Writer() {
  // First check self into system
  acquire(&lock);
  while ((AW + AR) > 0) {              // Is it safe to read?
    WW++;                              // No. Writers exit
    cond_wait(&ok_to_write, &lock);    // Sleep on cond var
    WW--;                              // No longer waiting
  }
  AW++;                                // Now we are active!
  release(&lock);
  // Perform actual read/write access
  access_database(read_write);
  // Now, check out of system
  acquire(&lock);
  AW--;                                // No longer active
  if (WW > 0)                          // Give priority to writers
    cond_signal(&ok_to_write);         // Wake up one writer
  else if (WR > 0)                     // Otherwise, wake up readers
    cond_broadcast(&ok_to_read);       // Wake up all readers
  release(&lock);
}
```

W1 gets signal (R3 waiting)
AR = 0, WR = 1, AW = 0, WW = 1

Simulation of readers/writers solution

```
Writer() {
  // First check self into system
  acquire(&lock);
  while ((AW + AR) > 0) {              // Is it safe to read?
    WW++;                              // No. Writers exit
    cond_wait(&ok_to_write, &lock);    // Sleep on cond var
    WW--;                              // No longer waiting
  }
  AW++;                                // Now we are active!
  release(&lock);
  // Perform actual read/write access
  access_database(read_write);
  // Now, check out of system
  acquire(&lock);
  AW--;                                // No longer active
  if (WW > 0)                          // Give priority to writers
    cond_signal(&ok_to_write);         // Wake up one writer
  else if (WR > 0)                     // Otherwise, wake up readers
    cond_broadcast(&ok_to_read);       // Wake up all readers
  release(&lock);
}
```

W1 gets signal (R3 waiting)

AR = 0, WR = 1, AW = 0, WW = 0

Simulation of readers/writers solution

```
Writer() {
  // First check self into system
  acquire(&lock);
  while ((AW + AR) > 0) {              // Is it safe to read?
    WW++;                              // No. Writers exit
    cond_wait(&ok_to_write, &lock);    // Sleep on cond var
    WW--;                              // No longer waiting
  }
  AW++;                                // Now we are active!
  release(&lock);
  // Perform actual read/write access
  access_database(read_write);
  // Now, check out of system
  acquire(&lock);
  AW--;                                // No longer active
  if (WW > 0)                          // Give priority to writers
    cond_signal(&ok_to_write);         // Wake up one writer
  else if (WR > 0)                     // Otherwise, wake up readers
    cond_broadcast(&ok_to_read);       // Wake up all readers
  release(&lock);
}
```

W1 gets signal (R3 waiting)

AR = 0, WR = 1, AW = 1, WW = 0

Simulation of readers/writers solution

W1 accessing database (R3 waiting)
AR = 0, WR = 1, AW = 1, WW = 0

```
Writer() {
  // First check self into system
  acquire(&lock);
  while ((AW + AR) > 0) {              // Is it safe to read?
    WW++;                              // No. Writers exit
    cond_wait(&ok_to_write, &lock);    // Sleep on cond var
    WW--;                              // No longer waiting
  }
  AW++;                                // Now we are active!
  release(&lock);
  // Perform actual read/write access
  access_database(read_write);
  // Now, check out of system
  acquire(&lock);
  AW--;                                // No longer active
  if (WW > 0)                          // Give priority to writers
    cond_signal(&ok_to_write);         // Wake up one writer
  else if (WR > 0)                     // Otherwise, wake up readers
    cond_broadcast(&ok_to_read);       // Wake up all readers
  release(&lock);
}
```

Simulation of readers/writers solution

```
Writer() {
  // First check self into system
  acquire(&lock);
  while ((AW + AR) > 0) {              // Is it safe to read?
    WW++;                              // No. Writers exit
    cond_wait(&ok_to_write, &lock);    // Sleep on cond var
    WW--;                              // No longer waiting
  }
  AW++;                                // Now we are active!
  release(&lock);
  // Perform actual read/write access
  access_database(read_write);
  // Now, check out of system
  acquire(&lock);
  AW--;                                // No longer active
  if (WW > 0)                          // Give priority to writers
    cond_signal(&ok_to_write);         // Wake up one writer
  else if (WR > 0)                     // Otherwise, wake up readers
    cond_broadcast(&ok_to_read);       // Wake up all readers
  release(&lock);
}
```

W1 finishes (R3 waiting)

AR = 0, WR = 1, AW = 0, WW = 0

Simulation of readers/writers solution

```
Writer() {
  // First check self into system
  acquire(&lock);
  while ((AW + AR) > 0) {              // Is it safe to read?
    WW++;                              // No. Writers exit
    cond_wait(&ok_to_write, &lock);    // Sleep on cond var
    WW--;                              // No longer waiting
  }
  AW++;                                // Now we are active!
  release(&lock);
  // Perform actual read/write access
  access_database(read_write);
  // Now, check out of system
  acquire(&lock);
  AW--;                                // No longer active
  if (WW > 0)                          // Give priority to writers
    cond_signal(&ok_to_write);         // Wake up one writer
  else if (WR > 0)                     // Otherwise, wake up readers
    cond_broadcast(&ok_to_read);       // Wake up all readers
  release(&lock);
}
```

W1 finishes (R3 waiting)

AR = 0, WR = 1, AW = 0, WW = 0

Simulation of readers/writers solution

```
Writer() {
  // First check self into system
  acquire(&lock);
  while ((AW + AR) > 0) {              // Is it safe to read?
    WW++;                              // No. Writers exit
    cond_wait(&ok_to_write, &lock);    // Sleep on cond var
    WW--;                              // No longer waiting
  }
  AW++;                                // Now we are active!
  release(&lock);
  // Perform actual read/write access
  access_database(read_write);
  // Now, check out of system
  acquire(&lock);
  AW--;                                // No longer active
  if (WW > 0)                          // Give priority to writers
    cond_signal(&ok_to_write);         // Wake up one writer
  else if (WR > 0)                     // Otherwise, wake up readers
    cond_broadcast(&ok_to_read);       // Wake up all readers
  release(&lock);
}
```

W1 finishes (R3 waiting)

AR = 0, WR = 1, AW = 0, WW = 0

Simulation of readers/writers solution

W1 signaling readers (R3 waiting)
AR = 0, WR = 1, AW = 0, WW = 0

```
Writer() {
  // First check self into system
  acquire(&lock);
  while ((AW + AR) > 0) {              // Is it safe to read?
    WW++;                              // No. Writers exit
    cond_wait(&ok_to_write, &lock);    // Sleep on cond var
    WW--;                              // No longer waiting
  }
  AW++;                                // Now we are active!
  release(&lock);
  // Perform actual read/write access
  access_database(read_write);
  // Now, check out of system
  acquire(&lock);
  AW--;                                // No longer active
  if (WW > 0)                          // Give priority to writers
    cond_signal(&ok_to_write);         // Wake up one writer
  else if (WR > 0)                     // Otherwise, wake up readers
    cond_broadcast(&ok_to_read);       // Wake up all readers
  release(&lock);
}
```

Simulation of readers/writers solution

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

R3 gets signal (no waiting threads)
AR = 0, WR = 1, AW = 0, WW = 0

Simulation of readers/writers solution

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

R3 gets signal (no waiting threads)
AR = 0, WR = 0, AW = 0, WW = 0

Simulation of readers/writers solution

R3 accessing database (no waiting threads)
AR = 1, WR = 0, AW = 0, WW = 0

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {           // Is it safe to read?
    WR++;                           // No. Writers exit
    cond_wait(&ok_to_read, &lock);  // Sleep on cond var
    WR--;                           // No longer waiting
  }
  AR++;                             // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                             // No longer active
  if (AR == 0 && WW > 0)            // No other active readers
    cond_signal(&ok_to_write);      // Wake up one writer
  release(&lock);
}
```

Simulation of readers/writers solution

R3 finishes (no waiting threads)
AR = 1, WR = 0, AW = 0, WW = 0

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

Simulation of readers/writers solution

R3 finishes (no waiting threads)
AR = 0, WR = 0, AW = 0, WW = 0

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

Questions

- Can readers starve? Consider Reader() entry code:

```
while ((AW + WW) > 0) {                 // Is it safe to read?
  WR++;                                 // No. Writers exit
  cond_wait(&ok_to_read, &lock);        // Sleep on cond var
  WR--;                                 // No longer waiting
}
AR++;
```

- What if we erase the condition check in Reader exit?

```
AR--;                                   // No longer active
if (AR == 0 && WW > 0)                  // No other active readers
  cond_signal(&ok_to_write);            // Wake up one writer
```

Questions

- Further, what if we turn the `signal()` into `broadcast()`?

```
AR--;                                   // No longer active
cond_broadcast((&ok_to_write)           // Wake up sleepers
```

- Finally, what if we use only one condition variable (call it "ok_continue") instead of two separate ones?
  - Both readers and writers sleep on this variable
  - Must use broadcast() instead of signal()

Code for a reader

```
Reader() {
  // First check self into system
  acquire(&lock);
  while ((AW + WW) > 0) {          // Is it safe to read?
    WR++;                          // No. Writers exit
    cond_wait(&ok_to_read, &lock); // Sleep on cond var
    WR--;                          // No longer waiting
  }
  AR++;                            // Now we are active!
  release(&lock);
  // Perform actual read-only access
  access_database(read_only);
  // Now, check out of system
  acquire(&lock);
  AR--;                            // No longer active
  if (AR == 0 && WW > 0)           // No other active readers
    cond_signal(&ok_to_write);     // Wake up one writer
  release(&lock);
}
```

Code for a writer

```
Writer() {
  // First check self into system
  acquire(&lock);
  while ((AW + AR) > 0) {              // Is it safe to read?
    WW++;                              // No. Writers exit
    cond_wait(&ok_to_write, &lock);    // Sleep on cond var
    WW--;                              // No longer waiting
  }
  AW++;                                // Now we are active!
  release(&lock);
  // Perform actual read/write access
  access_database(read_write);
  // Now, check out of system
  acquire(&lock);
  AW--;                                // No longer active
  if (WW > 0)                          // Give priority to writers
    cond_signal(&ok_to_write);         // Wake up one writer
  else if (WR > 0)                     // Otherwise, wake up readers
    cond_broadcast(&ok_to_read);       // Wake up all readers
  release(&lock);
}
```

Basic structure of *Mesa* monitor program

- Monitors represent the synchronization logic of the program
  - Wait if necessary
  - Signal when change something so any waiting threads can proceed

```
lock
while (need to wait) {
  condvar.wait();
}
unlock

do something so need to wait

lock
condvar.signal();
unlock
```

Check and/or update state variables
Wait if neccessary

Check and/or update state variables

Tips on concurrency coding

C-language support for synchronization

- Pretty straightforward, just functions to call

- But make sure you know all the code paths out of a critical section!

```
int Rtn() {
  acquire(&lock);
  …
  if (exception) {
    release(&lock);
    return errReturnCode;
  }
  …
  release(&lock);
  return OK;
}
```

Concurrency and synchronization in C

- Harder to track what to release with more locks

```
void Rtn() {
  lock1.acquire();
  …
  if (error) {
    lock1.release();
    return;
  }
  …
  lock2.acquire();
  …
  if (error) {
    lock2.release()
    lock1.release();
    return;
  }
  …
  lock2.release();
  lock1.release();
}
```

C-language support for synchronization

- Languages that support exceptions are problematic (easy to make a non-local exit without releasing lock)

```
void Rtn() {
  lock.acquire();
  …
  DoFoo();
  …
  lock.release();
}

void DoFoo() {
  …
  if (bad stuff) throw Exception;
  …
}
```

- An exception in `DoFoo()` will exit `Rtn()` without releasing the lock!

C-language support for synchronization

- Must catch all exceptions in critical sections
  - Catch exceptions, release lock, and re-throw exception:

```
void Rtn() {
  lock.acquire();
  try {
  …
  DoFoo();
  …
  } catch (…) {
    lock.release();
    throw;
  }
}
void DoFoo() {
  …
  if (exception) throw Exception;
  …
}
```

Much better pattern: C++ lock guards

```
#include <mutex>
int global_i = 0;  std::mutex global_mutex;

void safe_increment() {
  std::lock_guard<std::mutex> lock(global_mutex);
  …
  global_i++;
  // Mutex released in the destructor of “lock”
}
```

Python with Keyword

- More versatile than we show here (can be used to close files, connections, etc.)

```
lock = threading.Lock()
…
with lock: # Automatically calls acquire()
  some_var += 1
  …
# release() called however we leave the “with” block
```

Java synchronized keyword

- Every Java object has an associated lock:
  - Lock is acquired on entry and released on exit from a synchronized method
  - Lock is properly released if exception occurs inside a synchronized method
  - s

```
class Account {
  private int balance;
  // object constructor
  public Account (int initialBalance) {
    balance = initialBalance;
  }
  public synchronized int getBalance() {
    return balance;
  }
  public synchronized void deposit(int amount) {
    balance += amount;
  }
}
```

Java support for monitors

- Along with a lock, every object has a single condition variable associated with it

- To wait inside a synchronized method:
  - `void wait();`
  - `void wait(long timeout);`

- To signal while in a synchronized method:
  - `void notify();`
  - `void notifyAll();`

Where are we going with synchronization?

Programs	Shared Programs	
Higher-level API	Locks Monitors	Semaphores Send/Receive
Hardware	Load/Store Test&Set	Disable Ints Compare&Swap

- Implement various higher-level synchronization primitives using atomic operations

Your feedback is welcomed!



- https://forms.cloud.microsoft/r/Q6szydQLXY

Thanks